

```
import React, { useState, useEffect } from
'react';
import { signOut } from 'firebase/auth';
import { collection, addDoc, query, where,
onSnapshot, deleteDoc, doc, updateDoc }
from 'firebase/firestore';
import { auth, db } from '../firebase';
import { useNavigate } from 'react-router-
dom';
import '../styles/SellerDashboard.css';

function SellerDashboard({ user }) {
  const navigate = useNavigate();
  const [listings, setListings] =
useState([]);
  const [loading, setLoading] =
useState(false);
  const [userInfo, setUserInfo] =
useState(null);
  const [showForm, setShowForm] =
useState(false);
  const [formData, setFormData] =
useState({
    name: '',
    description: '',
    price: '',
    category: 'cosmetics'
  });
  const [error, setError] = useState('');
  const [success, setSuccess] =
```

```

useState('');

    const categories = ['cosmetics',
'clothing', 'books', 'food', 'accessories',
'other'];

    useEffect(() => {
        // Fetch user info
        const userQuery = query(collection(db,
'users'), where('email', '==',
user.email));
        const userUnsubscribe =
onSnapshot(userQuery, (snapshot) => {
            if (snapshot.docs.length > 0) {

setUserInfo(snapshot.docs[0].data());
            }
        });

        // Fetch user's listings
        const listingQuery =
query(collection(db, 'listings'),
where('sellerId', '==', user.uid));
        const listingUnsubscribe =
onSnapshot(listingQuery, (snapshot) => {
            const items = snapshot.docs.map(doc
=> ({ id: doc.id, ...doc.data() }));
            setListings(items);
        });

        return () => {
            userUnsubscribe();
            listingUnsubscribe();
        };
    }
  );
}

```

```

}, [user]));

const handleChange = (e) => {
  const { name, value } = e.target;
  setFormData(prev => ({
    ...prev,
    [name]: value
  }));
};

const handleAddListing = async (e) => {
  e.preventDefault();
  setError('');
  setSuccess('');

  if (!formData.name ||
    !formData.description || !formData.price) {
    setError('Please fill all fields');
    return;
  }

  setLoading(true);
  try {
    await addDoc(collection(db,
'listings'), {
      name: formData.name,
      description: formData.description,
      price: parseInt(formData.price),
      category: formData.category,
      sellerId: user.uid,
      sellerName: userInfo?.name ||
'Anonymous',
      sellerEmail: user.email,
      verifiedSeller:

```

```
userInfo?.verifiedSeller || false,
    status: 'active',
    createdAt: new Date(),
    views: 0,
    likes: 0
  });

  setSuccess('Listing created
successfully!');
  setFormData({ name: '', description:
'', price: '', category: 'cosmetics' });
  setShowForm(false);

  setTimeout(() => setSuccess(''),
3000);
} catch (err) {
  setError(err.message);
}
setLoading(false);
};

const handleDeleteListing = async
(listingId) => {
  if (!window.confirm('Are you sure? You
cannot undo this.')) return;

  try {
    await deleteDoc(doc(db, 'listings',
listingId));
    setSuccess('Listing deleted');
    setTimeout(() => setSuccess(''),
2000);
  } catch (err) {
    setError(err.message);
  }
}
```

```

    }
  };

  const handleLogout = async () => {
    await signOut(auth);
    navigate('/');
  };

  return (
    <div className="seller-dashboard">
      <div className="navbar">
        <div className="navbar-brand">MUBS
        <span>Seller Dashboard</span></div>
        <div className="navbar-actions">
          <button className="btn-small"
            onClick={() =>
              navigate('/marketplace')}>Browse</button>
          <button className="btn-small"
            onClick={handleLogout}>Logout</button>
        </div>
      </div>

      <div className="container">
        <div className="header">
          <h1>Your listings</h1>
          <p>Manage what you're selling on
            the MUBS marketplace</p>
        </div>

        {userInfo && (
          <div className="seller-info">
            <div>
              <strong>{userInfo.name}
            </strong> {userInfo.verifiedSeller ? '✓' :

```

```

'(Pending verification)'}
    </div>
    <div style={{ fontSize:
'0.9rem', color: '#6b7280' }}>{user.email}
</div>
  </div>
)}

{error && <div className="error-
message">{error}</div>}
{success && <div
className="success-message">{success}
</div>}

  <button
    className="btn-primary new-
listing-btn"
    onClick={() =>
setShowForm(!showForm)}
  >
    {showForm ? '× Cancel' : '+ New
listing'}
  </button>

  {showForm && (
    <form className="listing-form"
onSubmit={handleAddListing}>
      <div className="form-group">
        <label>Item name</label>
        <input
          type="text"
          name="name"
          value={formData.name}
          onChange={handleChange}

```

```

        placeholder="e.g. MAC
Lipstick Red"
        required
    />
</div>

<div className="form-group">
    <label>Description</label>
    <textarea
        name="description"
        value=
{formData.description}
        onChange={handleChange}
        placeholder="Describe what
you're selling"
        rows="3"
        required
    />
</div>

<div style={{ display: 'grid',
gridTemplateColumns: '1fr 1fr', gap: '1rem'
}}>

    <div className="form-group">
        <label>Price (UGX)</label>
        <input
            type="number"
            name="price"
            value={formData.price}
            onChange={handleChange}
            placeholder="45000"
            required
        />
    </div>

```

```

        <div className="form-group">
          <label>Category</label>
          <select name="category"
value={formData.category} onChange=
{handleChange}>
            {categories.map(cat => (
              <option key={cat}
value={cat}>

{cat.charAt(0).toUpperCase() +
cat.slice(1)}

              </option>
            ))}
          </select>
        </div>
      </div>

      <button type="submit"
className="submit-btn" disabled={loading}>
        {loading ? 'Posting...' :
'Post listing'}
      </button>
    </form>
  )}

  <div className="listings-section">
    <h2>Your items
({listings.length})</h2>

    {listings.length === 0 ? (
      <div className="empty-state">
        <p>You haven't listed
anything yet</p>

```



```

        <p style={{ fontSize:
'0.9rem' }}>Create your first listing to
start selling</p>
    </div>
    ) : (
        <div className="listings-
table">
            <div className="table-
header">
                <div>Item</div>
                <div>Price</div>
                <div>Category</div>
                <div>Posted</div>
                <div>Action</div>
            </div>
            {listings.map(item => (
                <div key={item.id}
className="table-row">
                    <div className="item-
name">{item.name}</div>
                    <div>UGX
{item.price.toLocaleString()}</div>
                    <div>{item.category}
                </div>
                <div style={{ fontSize:
'0.9rem', color: '#6b7280' }}>
{item.createdAt?.toDate().toLocaleDateStrin
g()}
                </div>
                <div>
                    <button
                        className="delete-
btn"

```

```

                                onClick={() =>
handleDeleteListing(item.id)}
                                >
                                    Delete
                                </button>
                            </div>
                        </div>
                    ))}
                </div>
            )}
        </div>
    </div>
</div>
);
}

export default SellerDashboard;

```